# TUNIO 트러블슈팅 기록

비전 없는 기타 자동 조율 로봇 — 2026-09-18 ~ 09-21 나흘간

정리 2026-09-21 · 근거는 커밋 해시와 이슈 번호로 달았다

# 이 문서를 읽는 법

나흘 동안 34건을 고쳤다. 그중 **조용히 틀리던 것**이 절반이다. 에러도 안 나고 동작도 성공으로 보고하면서 결과만 틀린 것들이다. 이 기록의 목적은 그것들을 남기는 데 있다.

각 항목은 네 칸이다 — **증상 / 원인 / 조치 / 남은 것**. 로봇이 없으면 재현할 수 없는 것이 많아, 원인을 어떻게 갈랐는지를 같이 적었다.

맨 뒤 12장에 **반복된 패턴 다섯 가지**를 따로 뽑았다. 개별 항목보다 그쪽이 다음 사고를 막는 데 쓸모 있을 것이다.

```
담당   정묵(페그)·성현(발현/피크)·정정인(마이크)·정동휘(서버/리뷰)
로봇   두산 M0609 + OnRobot RG2 / ROS 2 Jazzy / rclpy
```

# 1. 두산 드라이버를 그대로 믿으면 안 되는 자리

DSR_ROBOT2.py 는 파이썬 래퍼인데 **반환 규약이 함수마다 다르다.** 여기서만 네 번 속았다. 전부 예외 없이 조용히 틀린다.

## 1.1 조건 검사 반환이 뒤집혀 있었다

**증상** — 실물에서 로봇이 페그로 들어가지도 않고 그리퍼만 여닫고 끝났다. 사용자 표현 그대로 "이거 그냥 그리퍼를 닫았다 여는데?"

**원인** — 래퍼가 이렇게 돌려준다.

```
ret = 0 if (result.success == True) else -1
```

**조건 충족이 0, 미충족이 -1 이다.** 파이썬에서 0 은 거짓이고 -1 은 참이라 if api.check_position_condition(...) 으로 쓰면 정확히 반대로 돈다. 물러난 지점에서 "아직 도달 안 함"(-1, 참)을 "도달했다"로 읽고 곧바로 반환했다. 힘 제어가 한 번도 밀지 않았다.

**조치** — 뜻으로 옮기는 _cond_met() 을 두고 그 자리에 근거를 적었다. -1 은 "미충족"과 "호출 실패"를 겸해 못 가르므로 미충족으로 본다.

**근거** — 커밋 bad0875, 이슈 #31

## 1.2 min/max 가 "넘었나"가 아니라 "범위 안인가"였다

**증상** — 로봇이 페그에서 떨어진 채 가만히 있는데 곧바로 "들어가다 막혔다 (Y 축 2.0N)" 를 냈다.

**원인** — check_force_condition(axis, max=2.0) 이 "2N 을 넘었다"가 아니라 **"힘이 2N 이하다"**는 뜻이다. 허공이라 당연히 참이고, 그걸 막힘으로 읽었다.

같은 API 에 **두 번 속은 것이다** — 1.1 이 반환 규약, 이번이 인자 의미.

**조치** — check_position_condition · check_force_condition 을 **둘 다 걷어냈다.** 위치와 힘을 get_current_posx · get_tool_force(DR_BASE) 로 직접 읽어 우리가 판단한다. 줄 수는 비슷한데 무슨 일이 일어나는지가 화면에 보인다 — 1초마다 "진입 +12.3/100.0mm, Y 축 힘 0.4N" 을 찍는다.

**남은 것** — 힘은 절댓값으로 본다. 미는 방향에 따라 부호가 어느 쪽으로 나올지 모른다. 실측 후 부호를 확정하면 조일 수 있다.

**근거** — 커밋 76c2ae7

## 1.3 get_tcp() 가 성공하면 문자열, 실패하면 숫자를 준다

**증상** — 없다. 타입이 섞이는 것을 코드가 모른 채 문자열 비교를 했다.

**원인** — get_tcp() 는 정상이면 str, 응답이 없으면 -1(int)을 돌려준다. -1 을 이름으로 오해하고 통과시키면 안 된다.

**조치** — _read_named() 가 str 이 아니면 무조건 None 으로 본다. 회귀 테스트를 박았다.

**근거** — DSR_ROBOT2.py 6283~6382행, 커밋 788ca01

## 1.4 서비스가 예외로 실패하면 -1 이 아니라 UnboundLocalError 가 난다

**원인** — except 절이 로그만 찍고 ret 을 안 채운 채 return ret 을 탄다. 그대로 새어 나가면 부르는 쪽이 못 알아본다.

**조치** — _read_named() 가 예외를 잡아 MotionError 로 바꾼다.

## 1.5 get_* 응답의 success 는 컨트롤러가 true 로 하드코딩한다

**원인** — dsr_controller2.cpp 1940·1969행. **의미가 없는 필드다.** 판단은 info 문자열로만 해야 한다.

**조치** — 판정을 success 에서 떼어냈다.

## 1.6 spin_until_future_complete 에 타임아웃이 없다

**증상** — 디스커버리가 끝나기 전에 부르면 **영원히 멈춘다.** 화면에 아무것도 안 나오고 Ctrl+C 도 잘 안 먹는다.

**조치** — connect() 가 먼저 서비스를 기다린다. 성현이 pluck_dummy_test.py 에 만들어 둔 것을 올렸다. 반드시 DSR 이 쓰는 그 노드로 기다려야 한다.

**재발** — 3장 3.3 참조. 같은 함정에 목록만 다르게 해서 또 걸렸다.

**근거** — 커밋 52a7f9d, 이슈 #21

# 2. TCP · 좌표 기준 — 208mm 가 조용히 어긋나 있었다

## 2.1 connect() 가 set_tcp 실패를 버리고 있었다

**증상** — 없었다. 그게 문제다.

**원인** — 두산 API 는 실패를 예외가 아니라 -1 로 돌려주는데 반환값을 버렸다. 거부돼도 "연결됨"이 찍히고 그대로 움직였다. 페그 좌표가 GripperDA_v1(툴 Z 208mm) 기준이라 TCP 가 안 켜지면 **208mm 어긋난 곳으로 직선 진입한다.** 헤드스톡 쪽이다.

**왜 아무도 몰랐나** — 2026-09-19 페그 교시 때 펜던트에서 TCP 를 켜 둔 채로 했다. set_tcp 가 -1 로 거부돼도 좌표는 맞았다. 우연히 맞고 있었던 것이다.

**조치** — 판단 기준을 "set 이 성공했나"에서 **"지금 켜져 있는 것이 우리가 쓸 것인가"**로 바꿨다. set 을 항상 부르되 판정은 get 으로 한다. 둘 다 아니면 움직이기 전에 멈춘다.

**근거** — 커밋 52a7f9d 788ca01, 이슈 #26 #23 #21

## 2.2 한 YAML 에 208mm 어긋난 두 기준이 섞여 있었다

**증상** — 성현이 set_tcp 실험에 성공한 직후 로봇이 엉뚱한 곳으로 가서 강제 정지했다.

**원인** — posx 는 **읽을 때 켜져 있던 TCP 기준의 숫자다.** 페그 4개는 GripperDA_v1 을 켜고, 발현 4줄과 거치대는 끄고(플랜지) 교시했다. 같은 파일에 208mm 어긋난 두 기준이 들어 있었다. 실험이 성공해서 TCP 가 바뀐 순간, 발현 좌표가 가리키는 자리가 200mm 옮겨갔다.

**조치** — 세 가지를 같이 했다.

- 교시점마다 tcp: 로 기준을 적는다. robot.tcp 와 다르면 teach_points 가 **조회를 거부한다.** 조용히 움직이는 것보다 멈추는 게 낫다
- kinematics.shift_tcp() — 오프셋 [0,0,208,0,0,0] 이 순수 평행이동이라 **재교시 없이 계산으로 옮겨진다**
- TCP 는 GripperDA_v1 하나로 통일한다 (2026-09-21 합의)

**검증** — 발현 접촉점 4줄이 두 기준으로 다 남아 있어 서로 견줄 수 있었다. 변환한 Z 가 5·2·1번에서 교시값과 **1.2~1.4mm** 차이다. 6번만 6.4mm 인데 그 사이 재교시됐다(450d151).

**근거** — 커밋 be8a191 788ca01, 이슈 #21

## 2.3 set_tool/set_tcp/add_tcp 가 전부 -1 로 거부된다

**증상** — 로봇은 건강한데(robot_mode=1, 알람 없음, 조회·이동 정상) **설정을 바꾸는 계열만** 전부 거부된다. 새 이름을 만드는 add_tcp 도 -1 이다.

**갈라낸 것** — 이름 문제가 아니다. add_tcp('새이름', ...) 도 똑같이 실패하므로 "GripperDA_v1 이 없다"가 아니라 **설정 변경 권한 자체가 없다**는 뜻이다. 펜던트가 제어권을 쥐고 있는 쪽이 유력하다.

**조치** — 이 현장에서는 펜던트로 켜 두고 코드는 get 으로 확인만 한다. tools/tcp_diag.py 로 로봇을 안 움직이고 재현할 수 있다.

**남은 것** — 미해결. 이슈 #23 열려 있다. 배포용 코드는 set 을 생략하지 않으므로, 설정이 되는 로봇에서는 그대로 돈다.

## 2.4 pluck_dummy_test 가 3줄만 어긋난 채 돌고 있었다

**원인** — CONTACT_X 의 6번만 플랜지 기준이었다. STRINGS_TO_PLUCK 이 [6] 이라 안 터졌을 뿐, **4줄로 바꾸면 세 줄이 어긋났다.**

**조치** — 208mm 옮겨 기준을 맞추고 tool/tcp 이름도 실제 것으로 되돌렸다.

# 3. ROS 2 디스커버리와 도메인

## 3.1 브링업과 스크립트가 다른 방에서 서로를 찾고 있었다

**증상** — 서비스 없음: /dsr01/dsr_controller2/tool/get_current_tool (30초). 그런데 브링업은 정상이고 dsr01 서비스가 220개 떠 있다.

**원인** — 12:20 에 ~/.bashrc 에 3 PC 연동 설정이 추가됐다 (ROS_DOMAIN_ID=10, ROS_DISCOVERY_SERVER=172.24.2.65:11811, ROS_SUPER_CLIENT=TRUE). 브링업은 **그 전에 연 터미널이라 도메인 0**, 스크립트는 새 터미널이라 **도메인 10** 이었다.

**조치** — 도메인을 10 으로 통일하고 브링업도 새 터미널에서 띄운다. **모든 스크립트가 시작할 때 ROS_DOMAIN_ID 를 첫 줄에 찍는다.**

**남은 걱정** — ROS_DISCOVERY_SERVER 가 걸려 있으면 그 서버가 꺼진 날 **같은 PC 안의 노드끼리도 못 찾는다.** 로봇 PC 단독 테스트 절차가 필요하다.

**근거** — 이슈 #21

## 3.2 디스커버리 자체가 불안정하다 (원인 미확정)

같은 노드 생성 패턴으로 다섯 번 시도한 결과다.

```
1  DISCOVERY_SERVER 끔          129/220 에서 90초 넘게 멈춤
2  소켓 버퍼를 2GB 로           효과 없음 (OS 문제 아니었음)
3  ros2 daemon 재시작, 끈 상태  30초 동안 dsr01 서비스 0개
4  DISCOVERY_SERVER 켬          5초 만에 220~250개 전부 성공
5  같은 설정으로 재시도         136 에서 90초 넘게 멈춤
```

4번에서 성공했는데 5번에서 **같은 설정으로 재현이 안 됐다.** 게다가 그 디스커버리 서버는 TCP 연결 자체가 거부되어 실제로 떠 있지 않은 것으로 보이는데도, 환경변수를 켜는 것만으로 동작이 바뀐다(SIMPLE → CLIENT 모드 전환으로 추정).

**남은 것** — 미해결. 이슈 #23.

## 3.3 힘 제어 서비스만 대기 목록에서 빠져 있었다

**증상** — "이번엔 걍 그리퍼도 안 움직였음."

**원인** — 힘 제어 서비스는 접두사가 dsr_controller2/force/ 로 이동·조회 계열과 다른데, _WAIT_SERVICES 에 이동·조회만 들어 있었다. 1.6 과 **같은 함정이고 목록에 빠진 서비스만 달랐다.**

**조치** — 여섯 개를 목록에 넣었다. 그리고 어디서 멈췄는지 화면에 안 보이던 것도 같이 고쳤다 — peg_register 가 logger 를 안 넘겨 진행 로그가 전부 버려지고 있었다.

**정정** — 나중에 사용자의 터미널 출력으로 이게 **그 증상의 원인이 아니었음**이 드러났다. 실제 원인은 1.2 였다. 고친 것 자체는 독립적으로 맞아서 남겨 뒀다.

**근거** — 커밋 c577a81

# 4. 그리퍼 — 읽을 수 있는 것과 없는 것

## 4.1 "파지 감지를 못 읽는다"고 적어 뒀는데 틀렸다

**원인** — 토픽으로 내보내는 노드가 없을 뿐, Compute Box 가 **Modbus 로 그대로 내준다.** 설치본 onrobot_rg_control/.../comModbusTcp.py 의 getStatus 가 홀딩 레지스터 258~275 를 읽어 폭(0.1mm 단위)·파지 감지·동작 중을 뽑고 있다.

**조치** — tools/gripper_status.py. pymodbus 가 이 PC 에 없어서 소켓으로 직접 짰다. **기능 코드 3(읽기)만 보내므로 구조적으로 아무것도 못 쓴다.** 디지털 I/O 제어와 같이 켜 둬도 된다.

**중요** — 제어는 디지털 I/O, 읽기는 Modbus 다. **경로가 다르다.** 헷갈리지 말 것.

**근거** — 커밋 4e6b021

## 4.2 파지 감지 플래그는 못 믿는다 — 폭으로 판정한다

**증상** — 빈손일 때도 파지 감지(레지스터 11번)가 '있음'으로 나온다.

**조치** — 판정은 **폭으로만** 한다. 실측으로 문턱을 잡았다.

```
빈손 13.0mm   손잡이 두께(눕힘) 19.6mm   길이(세움) 32.8mm
16.3 미만 빈손 / 16.3~26.2 물림 / 26.2 초과 사점
```

같이 확인된 것 — **폭 레지스터 9번이 맞다.** 추정이었는데 실물과 일치했다.

**근거** — 커밋 03f1de0

## 4.3 페그 파지를 회전 토크로 확인하는 건 안 된다 (실측으로 부정)

**경위** — 9/19 에 "웜기어 자기잠금이 막는 방향은 반대라 우리가 돌릴 때 드는 저항은 읽힌다"고 적었다. **방향이 열려 있는 것과 신호가 잡히는 것은 다른 문제였다.**

**실측** — 6번 줄 허공 회차를 9/19 파지 회차와 견줬다.

```
회전 중 평균   파지 -0.3675 (흩어짐 0.110)   허공 -0.3449 (흩어짐 0.097)
```

차이 0.023 이 흩어짐의 1/4 이다. 기울기도 양쪽 다 없다. 계산과 맞는다 — 웜기어 14:1 을 지나면 줄 장력 몫이 손잡이에서 0.015 Nm 라 센서 잡음 0.11 Nm 에 묻힌다.

**도구가 거짓 보고를 했다** — "토크로 파지를 가릴 수 있다"고 찍는데, 정지 – 회전중 단차의 **크기만** 빼서 비교한다. 차이가 회전 저항이 아니라 **정지 baseline**(파지 +0.07 / 허공 +0.433)에서 왔다. 부호도 물리와 반대다 — 물면 저항이 늘어야 하는데 허공 단차가 더 컸다.

**살아 있는 단서** — 정지 baseline 차이 0.363 자체는 남아 있다. 돌리지 않고도 파지를 본다는 뜻이라 오히려 낫지만, 두 회차가 다른 날이고 9/19 기록에 팔 자세가 없어(손으로 옮긴 회차) 지금 근거로는 못 쓴다. **한 세션에서 허공·파지를 번갈아 2~3번씩 재야 갈린다.**

**결론** — Modbus 파지 감지가 1순위다.

**근거** — 커밋 a6ae5a6 de0a96e, 원자료 docs/robot_data/

# 5. 페그 파지 기하 — 열기 폭에 답이 없었다

이 항목은 나흘 중 가장 오래 걸렸고, **결론이 "코드로는 못 푼다"였다.** 측정과 추론이 다섯 번 뒤집혔으므로 순서대로 적는다.

## 5.1 90mm → 30mm

**증상** — 페그를 놓을 때 그리퍼가 가로로 벌어져 **옆 페그를 쳤다.**

**조치** — 웹로직 열기 프리셋을 30mm 로 내렸다. 90mm 는 반경 45mm 라 페그 간격 38mm 를 넘는다.

## 5.2 30mm 로는 세로로 선 손잡이를 못 지난다

**증상** — "2/3에서 페그를 완전 세로로 돌리는 게 30mm에서는 불가능해."

**원인** — 손잡이 길이를 모르고 정한 값이었다. 실측하니 길이 32.8mm 다.

## 5.3 최악은 90도가 아니라 대각선 59도다

이게 이 장의 핵심이다. 죔 방향에 필요한 폭은

```
p(θ) = 길이·sinθ + 두께·cosθ
```

라 **최대가 90도가 아니라 대각선이다.**

```
√(32.8² + 19.6²) = 38.2mm, θ = 59도.  90도(32.8mm)보다 5.4mm 넓다.
```

## 5.4 45mm 는 옆 페그를 친다 — 이웃 손잡이를 안 셌다

**증상** — 40mm 에서 "45mm여도 옆 페그는 안 건드림"을 확인받았는데, 올려 보니 **쳤다.**

**원인** — 이웃 페그의 손잡이가 **우리 쪽으로 뻗어온다.**

```
간격 38.0 − 이웃 손잡이 32.8/2 = 16.4  →  실효 21.6mm
45mm 열면 손끝이 22.5mm  →  0.9mm 초과. 친다
```

처음에 45 로 안 쳤던 건 **그때 이웃 손잡이가 다른 각이었기 때문이다.**

## 5.5 "물러나서 돌리면 45mm를 쓸 수 있다"는 틀렸다

한 번 이렇게 적었다가 스스로 정정했다. **폭 한계는 어디서 돌든 같다.** 결국 페그 앞에 벌어진 채로 도착해야 하기 때문이다.

다만 회전 중 휘두르는 위험은 별개로 실재한다. 페그 앞에서 45도를 돌리면 벌어진 손가락이 이웃 손잡이를 쓸고 간다. 그래서 접근점(100mm 뒤)까지 빠져서 돌린다.

## 5.6 창이 겹치지 않는다 — 폭으로는 답이 없다

41mm 도 42mm 도 쳤다. 38mm 에서 안 쳤다.

```
38.2mm 보다 커야   59도로 선 손잡이에 안 닿는다
38.0mm 이하여야    옆 페그를 안 친다
```

**창이 비어 있다. 더 잴 것도 없다.**

**그래서 다른 축으로 푼다** — 사람이 작업 전에 세로로 선 손잡이를 눕힌다. 조율 전이라 음이 틀어져도 된다. 빠뜨리면 로봇이 진입 중 접촉으로 알아채고 **어느 줄인지 짚어 주며** 멈춘다. 사람이 네 줄을 검사하는 게 아니다. 한 번 등록되면 각을 아니까 그 작업 내내 다시 안 걸린다.

**같이 걷어낸 것** — 사점 회전 재시도. 38mm 에서는 돌릴 공간 자체가 없어서 있으나 마나가 아니라 **틀린 동작**이었다.

**테스트로 묶었다** — 폭 안전 조건을 "open > 대각선"에서 **"덮지 못하면 requires_prealign 이 true 여야 한다"**로 바꿨다. 폭을 다시 건드릴 때 이 전제를 같이 지우는 일이 없게 했다.

**근거** — 커밋 76d4d4e 03f1de0 a6068a4 0e0c01d, 이슈 #31

## 5.7 여담 — 순응제어로 각을 맞추려던 계획도 접었다

처음 계획은 "순응제어를 켜고 그리퍼를 돌리면서 어떤 각이든 맞춰 잡는다"였다. **손목이 페그보다 1400배 뻣뻣해서 안 된다.** 순응제어의 용도는 각도 정렬이 아니라 **삽입축 밀림 방지**로 바뀌었다. 네 페그 접근축이 전부 base ±Y 라 Y 를 무르게 두면 걸렸을 때 물러나 기타를 안 민다.

# 6. 페그 각도 — 돌린 게 통째로 풀리던 문제

## 6.1 다시 들어갈 때 교시 자세로 정렬해 이전 회전을 풀어 버렸다

**증상** — 사용자 표현 그대로. "페그를 조금 돌리고 다시 더 돌릴 때 이전에 돌린 만큼 돈 상태로 들어가서 잡아야 하는데, 다시 그리퍼를 세로로 정렬하고 들어가서 **이전에 돌린 게 롤백되거든.**"

**원인** — 손잡이는 직전에 돌린 만큼 틀어져 있는데 그리퍼는 교시 자세로 들어간다. 작업 파지가 여유 파지(21mm)라 **페그가 그리퍼 쪽으로 끌려오면서** 직전 회전이 그대로 풀린다. 파지에도 성공하고 동작도 성공으로 보고한다.

**조치** — peg_state.PegState 가 페그별 손잡이 각을 들고 있는다. 교시 자세 기준 **상대** 각이고, Phase 0 등록값에서 시작해 조절할 때마다 누적한다. 손잡이가 180도 대칭이라 [-90, 90) 으로 접어 손목이 덜 돌게 했다.

**왜 경로가 안 바뀌나** — 반영은 파지 자세의 C 에 더하는 것으로 끝난다. intrinsic ZYZ 에서 마지막 Rz(C) 가 툴 Z 둘레 회전이고 툴 Z 자체는 A·B 로 정해지므로 **접근축도 손끝 위치도 안 바뀐다.** 경로는 그대로 두고 손가락 방향만 돈다.

**파일로 남긴다** — 노드가 죽었다 살아나도 Phase 0 를 다시 안 돈다. 대신 등록 시각을 같이 적고 max_age_sec(기본 4시간)를 넘으면 미등록으로 본다. **사람이 그사이 손으로 돌려놨으면 기록이 거짓말이 되기 때문이다.**

**근거** — 커밋 76d4d4e, 이슈 #31

## 6.2 물러날 때도 되돌리고 있었다

**원인** — 돌린 뒤 _retreat 가 **들어올 때 자세로** 물러나 손목이 역회전했다. 그리퍼를 열어도 페그가 아직 손가락 사이라 다시 돌려 버릴 수 있다.

**조치** — 다 돌았으면 돌아간 자세로, 도중에 멈췄으면 들어온 자세로 나간다. 도중 실패는 얼마나 돌았는지 모르기 때문이다. 같은 이유로 누적도 robot_controller 가 **성공한 뒤에만** 적는다.

## 6.3 들어갈 때 여는 동작이 빠져 있었다

**원인** — 나올 때는 _release() 가 open 프리셋으로 열었는데 **들어갈 때가 비어 있었다.** 직전 동작 상태 그대로 진입한다. 피크를 물고 있었으면 15mm 인 채로 들어가 손잡이를 정면으로 들이받는다.

**근거** — 커밋 8b25fa6

## 6.4 각 추적이 J6 전용인데 관절 번호를 안 봤다

**지적** — 정동휘, PR #36 리뷰 ④.

**원인** — peg_joint 가 확정값이 아니다. YAML 이 "실물 확인 후 확정"이고 teach_points 검증이 1~6 을 다 통과시킨다. 테스트가 실제로 peg_joint = 5 를 둔다. 그런데 robot_controller 가 add_turn 을 **관절 번호와 무관하게** 불러 J5 회전량을 C 변화량인 것처럼 누적하고, 다음 진입이 그 값을 C 에 더해 손목이 틀린 방향으로 돈다. **조용히 6.1 그 자체로 되돌아간다.**

**조치** — 국소 분기 대신 가정을 한 번 크게 선언했다. roll_deg 가 있는데 J6 이 아니면 MotionError 로 **움직이기 전에** 멈춘다.

**근거** — 커밋 f2a5008

## 6.5 Phase 0 가 도구에만 있고 노드는 안 불렀다

**원인** — register_peg 이 도구에만 있었다. 노드 경로에서는 각이 늘 None 이라 **경고만 찍고 교시 자세로 들어갔다.** 오늘 고친 각 추적이 노드에서는 놀고 있었다.

**조치** — 피크 집기 시점을 로봇이 스스로 판단하는 것과 같은 방식으로, 등록도 **필요해지는 순간** 한다. _ensure_registered() 가 미등록이면 그 자리에서 Phase 0 를 돈다. **인터페이스가 안 바뀐다.**

전용 Action(/robot/register_pegs)을 두는 길도 있었지만 팀 합의가 필요하고, 선례(피크)가 이미 이 방식이라 일관성을 택했다.

**모르면 돌리지 않는다** — 등록에 실패하면 goal 이 실패하고 메시지에 어느 줄인지 남는다.

**근거** — 커밋 5d0b6d8

# 7. 힘 제어와 순응 제어

## 7.1 기타를 밀고 들어갔다 — 세 값을 곱해보지 않았다

**증상** — "근데 이걸로 하니까 기타를 조금 밀어."

**원인** — 세 값을 따로 정하고 곱해보지 않았다.

```
순응 강성 Y   200 N/m = 0.2 N/mm
걸음          100mm / 8 = 12.5mm
한 걸음 힘    12.5 × 0.2 = 2.5N
문턱          8N        ← 영영 못 닿는다
```

접촉 감지가 **한 번도 안 걸리고** 끝까지 밀고 들어갔다. 강성을 더 낮추는 건 답이 아니다 — 낮출수록 힘이 늦게 쌓여 감지가 더 늦어진다.

**더 근본적으로** — 나눠 들어가던 것 자체가 우회였다. movel 이 끝날 때까지 안 돌아와서 중간에 볼 수가 없으니 쪼갠 것인데, 그건 **밀고 나서 확인**이라 원리상 이미 늦다.

**조치** — set_desired_force 로 접근축에 상한을 건다. 로봇이 **그 힘까지만 민다.** 감지가 아니라 제어다. 기타가 몇 N 에서 밀리는지 몰라도 된다 — "몇 N 에서 멈추나"가 아니라 "몇 N 까지만 허용하나"라서 낮게 잡으면 안전하다.

**시작값** (전부 실측 전 임시값) — insert_force_n 3.0 / contact_force_n 2.0 / arrive_tol_mm 2.0 / insert_timeout_sec 20.0

**근거** — 커밋 0206805

## 7.2 3N 으로 100mm 를 가면 느리기만 하다

**증상** — 사용자 지적. "3N으로 100mm 움직이려면 너무 오래 걸리지 않을까?"

**조치** — 진입을 2단으로 나눴다. 접근점에서 페그 앞 search_mm(기본 25mm) 까지는 위치 제어(movel)로 빠르게 가고, **거기서부터만 힘 제어로 더듬는다.** 힘 제어는 접촉 탐색용이지 이동용이 아니다.

**덤** — 순응 강성 200 N/m 에 3N 이면 평형 변위가 15mm 라, **100mm 를 힘 제어로 가려 했으면 애초에 못 갔을 가능성이 있다.**

**근거** — 커밋 a83bd18

## 7.3 --no-force 가 순응까지 꺼 버렸다

**지적** — 정동휘, PR #37 리뷰 ①.

**원인** — 측정 세션용으로 힘 제어를 끄는 길을 뒀는데, 순응 제어까지 같이 꺼졌다. 의도가 아니었다. 순응은 삽입축 밀림 방지라 **끌 이유가 없다.**

**조치** — 순응은 항상 켠다.

# 8. 피크 집기·반환

## 8.1 화음이 피크만 집고 죽는다

**원인** — strum_all 이 아직 NotImplementedError 인데 그 앞에 _ensure_pick() 이 붙어 있었다. dry_run 이 아니면 **진짜로 피크를 집은 뒤** 예외가 난다. 로봇은 피크를 쥔 채 멈추고 반환은 안 탄다. 사람이 가서 빼야 한다.

**피크 로직이 들어가면서 없던 문제가 생겼다.** 전에는 바로 예외였다.

**조치** — _ensure_pick() 을 빼고 TODO 를 남긴다. 구현할 때 한 줄 되돌리면 된다.

## 8.2 집기가 끊기면 다음에 기타 위에 떨어뜨린다

**원인** — pick_up 은 그리퍼를 **닫은 뒤** 기다렸다 들어 올린다. 그 사이 어디서든 끊기면 피크는 손에 있는데 _holding_pick 은 False 로 남는다 (대입이 맨 뒤라서). 그러면 다음 _ensure_pick() 이 pick_up() 을 다시 부르고, **그 첫 동작이 _release()** 다. 지금 있는 자리에서 손을 편다. 대기 자세면 기타 위고, 놓친 피크는 비전이 없어 못 찾는다.

**조치** — True/False 에 **None(모름)**을 더했다. 그리퍼를 닫기 **전에** None 으로 바꾸고 끝까지 가면 True 로 올린다. None 이면 집으러 가지 않고 멈춰서 사람을 부른다.

**갈린 판단** — 반대 방향(_release_pick)에서 정묵은 "일단 돌려놓는다"(빈손 왕복은 손해가 없으니까), 성현은 "거부하고 사람을 부른다"였다. **성현이 맞다** — 집기가 끊겼으면 팔이 어디 있는지도 모르는데 pick_return 은 지금 자리에서 거치대까지 직선으로 간다. 그 경로가 안전하다는 보장이 없다.

**근거** — 이슈 #34, 커밋 3a8e166

## 8.3 거치대에서 페그로 직행하는 경로는 아무도 안 봤다

**원인** — 2026-09-19 충돌 검증은 페그↔페그와 경유지뿐이었다. 반환 직후 팔이 거치대 위에 남는데, 그 자리에서 페그로 가는 movejx 는 검증 밖이다.

**조치** — 반환 뒤 _side 를 비워 다음 조절이 **무조건 경유지를 타게** 한다. 검증하려면 tools/transit_check.py from holder no-route 로 본다.

## 8.4 _holding_pick 이 모름으로 굳으면 복구가 노드 재시작뿐이다

**미해결.** 설계 결정 사항이라 이슈 #39 로 따로 올렸다.

# 9. 측정(오디오) 쪽

## 9.1 RMS 로는 방 소음을 못 거른다

**증상** — 실측에서 **방 소음 RMS 가 실제 연주와 완전히 겹쳤다.**

**조치** — 관문을 둘 더 걸었다 — FFT 피크 돌출 ≥20dB, 겹친 창 일치 폭 ≤12센트.

## 9.2 로봇 구동음이 유효 측정으로 통과했다

**증상** — 문턱 16dB 에서 **발현이 없는데도** 로봇 구동음 1194구간 중 35개(2.9%)가 유효 측정으로 통과했다. 그 값이 목표에서 **−133 ~ +92센트** 벗어나 있었다. 그대로 두면 서버가 그 값으로 페그를 크게 잘못 돌린다.

**조치** — 16 → 20dB. 대가로 약한 발현 통과율이 96% → 64% 로 떨어졌다.

**근거** — docs/로봇구동음_측정결과.md

## 9.3 "세게"의 기준은 RMS 가 아니라 돌출이다

**정정** — 한때 "RMS ≥ −18dBFS" 를 목표로 적었는데 틀렸다. 4줄 40회 실측에서 **RMS 와 돌출이 같이 가지 않았다** — 1번 E4 는 RMS −26dBFS 로 가장 약한데 돌출은 32~39dB, 6번 E2 는 RMS 가 높아도 돌출 최소 25.2dB 다. 방 소음 RMS 가 연주 RMS 와 겹치는 것과 같은 이유다.

**남은 위험** — 6번 E2 가 반대로 위험하다. 사람 발현 돌출 최소 25.2dB 에서 로봇 소음 3~8dB 를 빼면 **20dB 를 못 넘길 수 있다.** 이슈 #32 에서 판정한다.

## 9.4 짧게 여러 번 재면 저음현은 영영 안 모인다

**원인** — 원안은 "짧게 여러 번 독립 측정해 모이면 채택"이었는데 저음현에서 성립하지 않는다. **6번 줄은 1.0초로 자르면 그것만으로 50센트가 흔들린다.**

**조치** — "긴 창을 겹쳐서 여러 번" 재고 서로 일치하는지를 본다. 반환값은 전체 구간으로 내고(정밀도 최대) 겹친 창들은 신뢰 확인에만 쓴다. 측정 응답 목표도 2.0 → 2.2초로 올렸다.

# 10. 도구와 테스트가 스스로 거짓말한 것

이 장이 이 기록에서 가장 중요할 수 있다. **검증 수단이 틀리면 틀린 것을 통과시킨다.**

## 10.1 가짜 API 가 파이썬 진릿값을 흉내 내고 있었다

1.1 의 뒤집힌 조건 검사를 **테스트가 통과시켰다.** 가짜 API 가 실제 규약 (0=충족)이 아니라 파이썬 진릿값을 흉내 내고 있었기 때문이다. 코드와 테스트가 같은 오해를 공유하면 테스트는 아무것도 안 잡는다.

**조치** — 가짜를 실제 규약으로 고치고, "0 이 충족이고 -1 이 미충족이며 진릿값은 그 반대"라는 회귀 테스트를 박았다.

## 10.2 colcon test 가 0개를 돌리면서 통과로 보였다

**원인** — setup.py 에 tests_require=['pytest'] 가 없어 colcon 이 python3 -m unittest -v 로 폴백했고, 인자 없는 unittest 는 아무것도 탐색하지 않는다. **"NO TESTS RAN" 이 통과처럼 보였다.** 31개가 통과하는데 아무도 돌리지 않고 있었다.

**근거** — 이슈 #9, 커밋 b3cb7ce

## 10.3 지연 측정 도구가 실제로 6번 줄을 115도 풀고 있었다

**지적** — 정동휘, 이슈 #25.

**원인** — tools/action_latency.py 가 /robot/adjust_peg 에 5도씩 23회, **누적 115도**를 보냈다. AdjustPeg 는 읽기 전용이 아니다. 6번 줄 기준 −255센트, 82.41 → 71.11 Hz 로 탐색 범위 밖으로 나간다. 그러면 튜너가 그 줄을 아예 못 찾고, 장력이 25% 빠지면서 **넥이 펴져 손대지 않은 줄까지 틀어진다.**

**그때 막아 준 것은** robot_controller 의 dry_run 기본값 하나뿐이었고, 그것은 이 도구가 확인하지도 않는 **남의 노드**(다른 PC 일 수도 있는) 설정이었다.

**조치** — 기본값을 target self 로. 자기 Action 서버를 띄우고 거기에 잰다. target robot 은 먼저 dry_run 파라미터를 읽어 true 가 아니면 거부한다. **읽지 못해도 거부한다.** goal 각도도 5.0 → 0.1도로 낮췄다.

## 10.4 회전 분석 도구가 평평한 데이터에서 "저항 시작점"을 찾아냈다

**원인** — 최댓값의 절반을 넘는 첫 점을 찾는 식이라 **평평한 데이터에서도 첫 점이 걸린다.** "저항이 붙기 시작한 지점 2.5도"라고 보고했다.

**조치** — 앞뒤 1/3 의 차이를 흩어짐과 견주도록 고쳤다. 4.3 의 판정 오류도 같은 계열이다.

## 10.5 팔 모양(sol) 자동 선택이 한 도구에만 빠져 있었다

**증상** — 2026-09-19 실물에서 경유지→6번 접근점 경로가 순회 때와 달라져 **부딪힐 뻔했다.**

**원인** — peg_turn_check 가 sol 을 안 고르고 peg.sol(=null)을 그대로 넘겼다. 그러면 로봇이 경유지의 현재 자세를 따라가는데, **경유지가 특이점이라 그 값이 모호해서** 갈 때마다 팔 모양이 달라진다.

**조치** — 선택 로직을 arm_pose.gather/choose 로 빼서 네 도구가 같이 쓴다. **구현이 하나라 한쪽만 고쳐지는 일이 없다.**

# 11. 협업에서 생긴 것

## 11.1 같은 이슈를 두 사람이 따로 고쳤다

**경위** — 이슈 #34(피크 집기·반환 안전성)를 성현이 15:50 에, 정묵이 16:33 에 각자 구현했다. 브랜치가 갈려 있어 서로 몰랐다.

**해소** — 성현 것으로 갔다. 피크 로직이 성현 영역이고, 갈린 세 군데 전부 성현 쪽이 나았다(8.2 참조). 정묵 커밋은 revert 했다.

**교훈** — 이슈에 "지금 이거 하는 중"을 적는 비용이 중복 구현보다 싸다.

## 11.2 PR 을 먼저 머지한 쪽이 상대 브랜치에 충돌을 만든다

**경위** — 성현이 06:51 에 PR 을 열었는데 그 무렵 정묵의 PR 이 머지되면서 같은 YAML 을 건드렸다. 성현은 **갱신된 좌표를 못 본 상태로** 판단하게 됐고, 충돌을 자기 쪽으로 풀면 09-20 실기 검증본이 09-19 유래본으로 되돌아간다.

**조용히 틀리는 지점** — stroke_sign 4개가 같이 사라지는데, getattr(pose, "stroke_sign", 1) or 1 이라 **없으면 +1** 이다. 고음 쪽 스트로크 방향이 **에러 없이** 뒤집힌다.

**남은 것** — 충돌 해소 시 main 쪽 stroke_sign 4개와 search_mm 을 살려야 한다.

## 11.3 웹에서 고친 값을 코드가 알 방법이 없다

RG2 의 폭·힘은 **OnRobot 웹로직에서 정하고 코드는 프리셋을 고를 뿐이다.** 웹에서 값을 고치면 teach_points.yaml 의 기록도 같이 고쳐야 하는데, **어긋나도 로봇은 그대로 움직인다.**

실제로 어긋나 있다 — close 프리셋이 기록은 15mm 인데 실제로 13.0mm 까지 닫힌다.

# 12. 반복된 패턴 다섯 가지

개별 항목보다 이쪽이 다음 사고를 막는 데 쓸모 있다.

### ① 반환값을 버리면 조용히 틀린다

두산 API 는 실패를 **예외가 아니라 숫자로** 돌려준다. 2.1 이 이것 하나로 208mm 오동작 직전까지 갔다. 그리고 그 숫자의 규약이 함수마다 다르다(1장 전체).

**대응** — set 의 반환이 아니라 get 의 결과로 판정한다. 뜻을 옮기는 함수를 두고 그 자리에 근거 줄번호를 적는다.

### ② 우연히 맞고 있는 것은 고쳐진 게 아니다

펜던트에 TCP 가 켜져 있어서 set_tcp 실패를 아무도 몰랐다(2.1). STRINGS_TO_PLUCK 이 [6] 이라 3줄 어긋남이 안 터졌다(2.4).

**대응** — "지금 되고 있다"와 "되게 만들어져 있다"를 구분해서 적는다.

### ③ 검증 수단이 틀리면 틀린 것을 통과시킨다

가짜 API 가 파이썬 진릿값을 흉내 내서 뒤집힌 코드를 통과시켰다(10.1). 테스트가 0개 돌면서 통과로 보였다(10.2). 분석 도구가 평평한 데이터에서 없는 신호를 찾아냈다(10.4, 4.3).

**대응** — 가짜는 **실제 규약**을 흉내 낸다. 도구의 판정은 원자료와 함께 본다.

### ④ 계산과 실측이 갈리면 실측이 이긴다 — 단, 실측 조건을 적어야 한다

손잡이 대각선을 22mm 로 짐작했는데 실측은 38.2mm 였다(5.3). 45mm 가 "안 친다"고 확인됐는데 나중에 쳤다 — **그때 이웃 손잡이가 다른 각이었다**(5.4). 토크로 파지를 본다는 계산이 실측으로 부정됐다(4.3).

**대응** — 측정에는 **그때의 조건**을 같이 적는다. 팔 자세를 안 적어서 9/19 회차를 못 쓰게 된 일이 실제로 있었다.

### ⑤ 모르면 멈춘다

이 프로젝트에는 비전이 없어서 **놓친 것을 되찾을 방법이 없다.** 그래서 _holding_pick 의 None(8.2), teach_points 의 기준 불일치 거부(2.2), Phase 0 실패 시 goal 실패(6.5)가 전부 같은 결정이다.

조용히 진행하는 것보다 **어느 줄인지 짚으며 멈추는 것**이 낫다.

# 부록 · 아직 안 풀린 것

- **set_tool/set_tcp/add_tcp 전부 거부** — 펜던트 제어권 의심. 이슈 #23
- **디스커버리 불안정** — 같은 설정으로 재현 안 됨. 이슈 #23
- **set_desired_force 가 실제로 미는지** — 실물 미검증. contact_force_n 도 임시값
- **페그 파지 확인 수단** — Modbus 폭이 1순위. 정지 baseline 은 한 세션 반복 측정이 필요하다. 이슈 #8 #13
- **_holding_pick 모름 상태의 복구 경로** — 지금은 노드 재시작뿐. 이슈 #39
- **로봇 발현의 돌출 문턱** — 로봇이 튕기고 로봇이 움직이는 조건의 실측이 없다. 6번 E2 가 위험하다. 이슈 #32
- **발현↔페그 자세 간섭** — 경유지는 페그 쪽만 검증됐다. 통합 실기에서 본다
- **취소(cancel)** — on_cancel 이 수락만 하고 로봇은 안 선다. move_stop/move_pause/move_resume 이 컨트롤러에는 있는데 DSR_ROBOT2.py 에는 없어 raw ROS 서비스로 불러야 한다. 정지 모드(0~3)는 사람이 정해야 한다